

O array

0 array

```
a = np.arange(12)
```

0 array

```
a = np.arange(12)
```

0	1	2	3	4	5	6	7	8	9	10	11
---	---	---	---	---	---	---	---	---	---	----	----

0 array

```
a = np.arange(12)
```

0	1	2	3	4	5	6	7	8	9	10	11
---	---	---	---	---	---	---	---	---	---	----	----

doze números, um tipo só, lado a lado na memória

0 array

```
a = np.arange(12)
```

0	1	2	3	4	5	6	7	8	9	10	11
---	---	---	---	---	---	---	---	---	---	----	----

doze números, um tipo só, lado a lado na memória

```
a.shape -> (12,)
```

0 array

```
m = a.reshape(3, 4)
```

0	1	2	3	4	5	6	7	8	9	10	11
---	---	---	---	---	---	---	---	---	---	----	----

doze números, um tipo só, lado a lado na memória

0 array

```
m = a.reshape(3, 4)
```

0	1	2	3
4	5	6	7
8	9	10	11

mesmos doze números, outra forma.
Nenhum dado foi copiado.

0 array

```
m = a.reshape(3, 4)
```

0	1	2	3
4	5	6	7
8	9	10	11

mesmos doze números, outra forma.
Nenhum dado foi copiado.

`m.shape` -> (3, 4)

Vetorização

$$m * 2$$

0	1	2	3
4	5	6	7
8	9	10	11

Vetorização

$$m * 2$$

0	2	4	6
8	10	12	14
16	18	20	22

Vetorização

$$m * 2$$

0	2	4	6
8	10	12	14
16	18	20	22

nenhum `for`: a operação vale para o array inteiro,
e roda em código compilado

Vetorização

$$m * 2$$

0	1	2	3
4	5	6	7
8	9	10	11

nenhum `for`: a operação vale para o array inteiro,
e roda em código compilado

Broadcasting

```
m + np.array([10, 20, 30, 40])
```

0	1	2	3
4	5	6	7
8	9	10	11

Broadcasting

```
m + np.array([10, 20, 30, 40])
```

10	20	30	40
0	1	2	3
4	5	6	7
8	9	10	11

Broadcasting

```
m + np.array([10, 20, 30, 40])
```

10	20	30	40
10	21	32	43
14	25	36	47
18	29	40	51

Broadcasting

```
m + np.array([10, 20, 30, 40])
```

10	21	32	43
14	25	36	47
18	29	40	51

a forma menor se ESTICA para casar com a maior,
sem você repetir o vetor três vezes

Broadcasting

```
m + np.array([10, 20, 30, 40])
```

0	1	2	3
4	5	6	7
8	9	10	11

a forma menor se ESTICA para casar com a maior,
sem você repetir o vetor três vezes

Máscara booleana

$$m > 5$$

0	1	2	3
4	5	6	7
8	9	10	11

Máscara booleana

$m > 5$

False	False	False	False
False	False	True	True
True	True	True	True

Máscara booleana

$m > 5$

False	False	False	False
False	False	True	True
True	True	True	True

comparar NÃO devolve um booleano:
devolve um array de booleanos,
do mesmo tamanho

Máscara booleana

```
m[m > 5]
```

False	False	False	False
False	False	True	True
True	True	True	True

comparar NÃO devolve um booleano:
devolve um array de booleanos,
do mesmo tamanho

Máscara booleana

```
m[m > 5]
```

False	False	False	False
False	False	6	7
8	9	10	11

comparar NÃO devolve um booleano:
devolve um array de booleanos,
do mesmo tamanho

Máscara booleana

`m[m > 5]`

False	False	False	False
False	False		

comparar NÃO devolve um booleano:
devolve um array de booleanos,
do mesmo tamanho

6	7	8	9	10	11
---	---	---	---	----	----

Máscara booleana

```
m[m > 5]
```

False	False	False	False
False	False		

sobra um array 1D só com os que passaram.
a forma da matriz se perde

6	7	8	9	10	11
---	---	---	---	----	----

axis

```
m.sum(axis=0)
```

0	1	2	3
4	5	6	7
8	9	10	11

axis

```
m.sum(axis=0)
```

**axis diz qual eixo DESAPARECE,
não qual sobra**

0	1	2	3
4	5	6	7
8	9	10	11

axis

```
m.sum(axis=0)
```

**axis diz qual eixo DESAPARECE,
não qual sobra**

0	1	2	3
4	5	6	7
8	9	10	11
12	15	18	21

axis

```
m.sum(axis=0)
```

**axis diz qual eixo DESAPARECE,
não qual sobra**

0	1	2	3
4	5	6	7
8	9	10	11

12	15	18	21
----	----	----	----

(3, 4) -> (4,)

axis

```
m.sum(axis=1)
```

**axis diz qual eixo DESAPARECE,
não qual sobra**

0	1	2	3
4	5	6	7
8	9	10	11

axis

```
m.sum(axis=1)
```

**axis diz qual eixo DESAPARECE,
não qual sobra**

0	1	2	3	6
4	5	6	7	22
8	9	10	11	38

axis

```
m.sum(axis=1)
```

**axis diz qual eixo DESAPARECE,
não qual sobra**

0	1	2	3	6
4	5	6	7	22
8	9	10	11	38

$(3, 4) \rightarrow (3,)$

axis

```
m.sum(axis=1)
```

axis=0 some com as **LINHAS**
axis=1 some com as **COLONAS**

0	1	2	3	6
4	5	6	7	22
8	9	10	11	38

(3, 4) -> (3,)

Um tipo só

Um tipo só

```
np.array([1, 2, 3])
```

1	2	3
---	---	---

dtype: int64

Um tipo só

```
np.array([1, 2, 3])
```

1	2	3
---	---	---

dtype: int64

um array tem UM dtype, para todos os elementos.
É isso que deixa a memória contígua, e a conta rápida.

Um tipo só

```
np.array([1, 2, 3])
```

1	2	3
---	---	---

dtype: int64

```
np.array([1, np.nan, 3])
```

1.0	nan	3.0
-----	-----	-----

dtype: float64

um array tem UM dtype, para todos os elementos.
É isso que deixa a memória contígua, e a conta rápida.

Um tipo só

```
np.array([1, 2, 3])
```

1	2	3
---	---	---

dtype: int64

```
np.array([1, np.nan, 3])
```

1.0	nan	3.0
-----	-----	-----

dtype: float64

NaN é um FLOAT. Um só já promove o array inteiro, e é por isso que, no pandas, coluna de inteiros com nulo vira float64.

Um tipo só

```
np.array([1, 2, 3])
```

1	2	3
---	---	---

dtype: int64

```
np.array([1, np.nan, 3])
```

1.0	nan	3.0
-----	-----	-----

dtype: float64

e um único NaN contamina a conta inteira

Um tipo só

```
np.array([1, 2, 3])
```

1	2	3
---	---	---

dtype: int64

```
np.array([1, np.nan, 3])
```

1.0	nan	3.0
-----	-----	-----

dtype: float64

```
np.array([1., np.nan, 3.]).sum()    ->  nan  
np.nansum([1., np.nan, 3.])        ->  4.0
```

e um único NaN contamina a conta inteira

Um tipo só

```
np.array([1, 2, 3])
```

1	2	3
---	---	---

dtype: int64

```
np.array([1, np.nan, 3])
```

1.0	nan	3.0
-----	-----	-----

dtype: float64

```
np.array([1., np.nan, 3.]).sum()    -> nan  
np.nansum([1., np.nan, 3.])       -> 4.0
```

a família nan* existe para isso

e um único NaN contamina a conta inteira

Números aleatórios

Números aleatórios

```
rng = np.random.default_rng(42)
```

Números aleatórios

```
rng = np.random.default_rng(42)
```

```
rng.random(4)
```

0.774	0.439	0.859	0.697
-------	-------	-------	-------

Números aleatórios

```
rng = np.random.default_rng(42)
```

```
rng.random(4)
```

0.774	0.439	0.859	0.697
-------	-------	-------	-------

floats entre 0 e 1

Números aleatórios

```
rng.integers(1, 7, 5)
```

```
rng.integers(1, 7, 5)
```

1	5	4	3	3
---	---	---	---	---

inteiros de 1 a 6:
o fim é exclusivo

Números aleatórios

```
rng.normal(100, 15, 4)
```

```
rng.normal(100, 15, 4)
```

104.6	84.4	111.3	114.1
-------	------	-------	-------

de uma normal,
média 100 e desvio 15

Números aleatórios

```
rng.choice(["loja", "site"],  
           5, p=[0.3, 0.7])
```

```
rng.choice(["loja", "site"],
```

site	site	site	site	loja
------	------	------	------	------

sorteio com
probabilidade declarada

Números aleatórios

```
np.random.default_rng(7).random(3)
```

Números aleatórios

```
np.random.default_rng(7).random(3)
```

duas execuções, semente 7

0.625	0.897	0.776
-------	-------	-------

0.625	0.897	0.776
-------	-------	-------

Números aleatórios

```
np.random.default_rng(7).random(3)
```

duas execuções, semente 7

0.625	0.897	0.776
-------	-------	-------

0.625	0.897	0.776
-------	-------	-------

**mesma semente,
mesmos números.**

Números aleatórios

```
np.random.default_rng(7).random(3)
```

duas execuções, semente 7

0.625	0.897	0.776
-------	-------	-------

0.625	0.897	0.776
-------	-------	-------

**mesma semente,
mesmos números.**

sem semente, cada execução dá outro resultado, e você não consegue reproduzir nem depurar.

Números aleatórios

```
np.random.default_rng(7).random(3)
```

duas execuções, semente 7

0.625	0.897	0.776
-------	-------	-------

0.625	0.897	0.776
-------	-------	-------

**mesma semente,
mesmos números.**

sem semente, cada execução dá outro resultado, e você não consegue reproduzir nem depurar.

`np.random.seed()` é a API antiga.
Use `default_rng()`.

Álgebra linear

Álgebra linear

A (2, 3)

1	2	3
4	5	6

B (3, 2)

7	8
9	10
11	12

Álgebra linear

A @ B

B (3, 2)

A (2, 3)

1	2	3
4	5	6

7	8
9	10
11	12

Álgebra linear

A @ B

A (2, 3)

1	2	3
4	5	6

B (3, 2)

7	8
9	10
11	12

$(2, 3) @ (3, 2) \rightarrow (2, 2)$

Álgebra linear

A @ B

A (2, 3)

1	2	3
4	5	6

B (3, 2)

7	8
9	10
11	12

$(2, 3) @ (3, 2) \rightarrow (2, 2)$

os de DENTRO têm de bater;
os de FORA viram a forma do resultado

Álgebra linear

A @ B

A (2, 3)

1	2	3
4	5	6

B (3, 2)

7	8
9	10
11	12

(2, 2)

58	64
139	154

$(2, 3) @ (3, 2) \rightarrow (2, 2)$

os de DENTRO têm de bater;
os de FORA viram a forma do resultado

Álgebra linear

B @ B

A (2, 3)

1	2	3
4	5	6

B (3, 2)

7	8
9	10
11	12

$(2, 3) @ (3, 2) \rightarrow (2, 2)$

os de DENTRO têm de bater;
os de FORA viram a forma do resultado

Álgebra linear

B @ B

B (3, 2)

A (2, 3)

1	2	3
4	5	6

7	8
9	10
11	12

ValueError

(3, 2) @ (3, 2)

os de DENTRO têm de bater;
os de FORA viram a forma do resultado

Álgebra linear

B @ B

B (3, 2)

A (2, 3)

1	2	3
4	5	6

7	8
9	10
11	12

ValueError

2 e 3 não batem:
o produto não existe

(3, 2) @ (3, 2)

os de DENTRO têm de bater;
os de FORA viram a forma do resultado

Álgebra linear

A.T

A (2, 3)

1	2	3
4	5	6

(2, 3) $\rightarrow$ (3, 2)

os de DENTRO têm de bater;
os de FORA viram a forma do resultado

Álgebra linear

A . T

A . T (3, 2)

1	4
2	5
3	6

(2, 3) -> (3, 2)

os de DENTRO têm de bater;
os de FORA viram a forma do resultado

Álgebra linear

```
M = np.array([[2, 1],  
              [1, 3]])  
b = np.array([7, 11])  
np.linalg.solve(M, b)
```

Álgebra linear

```
M = np.array([[2, 1],  
              [1, 3]])  
b = np.array([7, 11])  
np.linalg.solve(M, b)
```

$$\begin{array}{rcl} 2x + y & = & 7 \\ x + 3y & = & 11 \end{array}$$

Álgebra linear

```
M = np.array([[2, 1],  
              [1, 3]])  
b = np.array([7, 11])  
np.linalg.solve(M, b)
```

x, y

2.0	3.0
-----	-----

$$\begin{array}{rcl} 2x + y & = & 7 \\ x + 3y & = & 11 \end{array}$$

Álgebra linear

```
M = np.array([[2, 1],  
              [1, 3]])  
b = np.array([7, 11])  
np.linalg.solve(M, b)
```

$$\begin{array}{rcl} 2x + y & = & 7 \\ x + 3y & = & 11 \end{array}$$

x, y

2.0	3.0
-----	-----

```
np.linalg.det(M)    -> 5.0  
np.linalg.inv(M)    -> a inversa  
M @ solucao         -> confere o resultado
```

Álgebra linear

```
M = np.array([[2, 1],  
              [1, 3]])  
b = np.array([7, 11])  
np.linalg.solve(M, b)
```

$$\begin{array}{rcl} 2x + y & = & 7 \\ x + 3y & = & 11 \end{array}$$

x, y

2.0	3.0
-----	-----

```
np.linalg.det(M)    -> 5.0  
np.linalg.inv(M)    -> a inversa  
M @ solucao         -> confere o resultado
```

**solve é melhor que inv @ b:
mais preciso e mais rápido.**

Quatro ideias

Quatro ideias

um dtype

memória contígua, e é daí que vem a velocidade `a.dtype`

Quatro ideias

um dtype

memória contígua, e é daí que vem a velocidade

`a.dtype`

vetorização

a operação vale para o array inteiro

`m * 2`

Quatro ideias

um dtype

memória contígua, e é daí que vem a velocidade

`a.dtype`

vetorização

a operação vale para o array inteiro

`m * 2`

broadcasting

a forma menor estica para casar

`m + [10, 20, 30, 40]`

Quatro ideias

um dtype

memória contígua, e é daí que vem a velocidade

```
a.dtype
```

vetorização

a operação vale para o array inteiro

```
m * 2
```

broadcasting

a forma menor estica para casar

```
m + [10, 20, 30, 40]
```

axis

diz qual eixo desaparece

```
m.sum(axis=0)
```

Quatro ideias

um dtype

memória contígua, e é daí que vem a velocidade

```
a.dtype
```

vetorização

a operação vale para o array inteiro

```
m * 2
```

broadcasting

a forma menor estica para casar

```
m + [10, 20, 30, 40]
```

axis

diz qual eixo desaparece

```
m.sum(axis=0)
```